

1) Explain the concept of persistence in I/O devices and compare polling with interrupt-driven I/O techniques and demonstrate how interrupt handling is implemented for device management in an operating system.

A) Persistence in I/O devices refers to the ability of a device to retain data even after power loss. Examples include hard drives and SSDs, which store files persistently, unlike RAM.

Polling (vs) Interrupt-driven I/O:-

Feature	Polling	Interrupt-driven I/O
CPU repeatedly checks device status	Device sends interrupt signal to CPU	CPU wastefully loops
Efficiency	CPU handles other tasks until interrupted	Simple, low-speed devices
Real-time	CPU Load	High
	Low	

Interrupt handling in OS:-

- Device sends an interrupt signal via an interrupt-request line.
- CPU pauses current execution and invokes an interrupt service routine (ISR).
- ISR identifies the device, processes the request, and resumes normal execution.

• OS uses interrupt vector tables to map devices to their handlers.

2) Apply various device management techniques used by an operating system to optimize I/O performance. Illustrate how methods such as buffering, caching, spooling, and scheduling enhance efficiency during device communication and data transfer.

A) Operating System use several strategies to improve I/O performance:

- Buffering: Temporarily stores data in memory while transferring between devices. Reduces wait time and allows overlapping of computations and I/O.
Ex: keyboard input buffered before being processed.
- Caching: Stores frequently accessed data in fast memory. Speeds up read operations.
Ex: Disk cache stores recently read blocks.
- Spooling: Queues data for devices that can't handle simultaneous requests.
Ex: print jobs are spooled to disk that can't handle simultaneous requests.
- Scheduling: Determines the order of I/O requests to optimize throughput and reduce latency.
- Algorithms include FCFS (First-Come-First-Serve), SSTF (Shortest seek time first), and SCAN.

3) Illustrate the concepts of a canonical device by applying its characteristics to a real-world example. Describe how a typical operating system interacts with such a device through device users and I/O operations, highlighting the standard input/output behaviour.

A) Canonical device is a standard interface that abstracts device-specific details. It provides consistent I/O behaviour across systems.

Ex: Terminal (keyboard + display)

- Acts as a character-oriented device.
 - Supports standard input/output operations (stdin, stdout).
 - OS interacts via device drivers that translate system calls (read(), write()) into hardware-specific instructions.
- OS interaction:
- Application calls read() → OS routes to terminal driver.
 - Driver collects input, applies canonical processing (e.g., line editing).
 - Output via write() is sent to display buffer.

4) Explain how data is retained in Persistent I/O devices after a System shutdown and discuss Common causes of data corruption in disk storage along with how the operating System uses check sums and parity for error detection.

A) Persistent I/O devices like HDDs, SSDs, and flash drives retain data even after the System shuts down, because they use non-volatile memory. Unlike RAM, which loses data when power is off, non-volatile storage preserves bits using physical.

- Hard Disk Drives:- Store data magnetically on spinning platters. Magnetic orientation of bits remains unchanged without power.

- Solid-State Drives:- Use NAND flash memory, where data is stored in floating-gate transistors that trap electrons.

- Flash drives:- Similar to SSDs, they use flash memory to retain data without needing continuous power.

⇒ Common causes of data corruption in disk storage.

- Power failures:- File writing incomplete → Corrupt file.

- Bad Sectors:- Disk area becomes unreadable.

- Malware:- Deletes (or) changes data

- File system errors:- Indexing (or) metadata issues

- Physical damage:- Drops, heat, (or) wear

⇒ Uses check sums and parity for Error detection

- Operating systems and Storage Systems use checksums & parity bits

⇒ Check Sums:-

- A checksum is a calculated value based on contents of a data block.

- If they differ, it signals corruption.

⇒ Parity Bits:-

- A parity bit is added to a set of bits to ensure even (or) odd parity.

- Helps detect single-bit errors in memory (or) disk blocks.

5) Given the following queue: 55, 58, 39, 18, 90, 160, 150, 38, 184 with the Read-Write head initially at the track 100 and the tail track being at 199. Apply the following Disk Scheduling algorithms and calculate the total head movement a) SCAN b) C-SCAN

A) Queue: 55, 58, 39, 18, 90, 160, 150, 38, 184

Initial head: 100

Tail track: 199

a) SCAN (Elevator Algorithm):

- Head moves towards end (199), servicing requests on the way.
- Then reverses direction and services remaining requests.

Sorted Queue: 18, 38, 39, 55, 58, 90, 150, 160, 184

Right of 100: 150, 160, 184

Left of 100: 90, 58, 55, 39, 38, 18

Movement: $100 \rightarrow 150 \rightarrow 160 \rightarrow 184 \rightarrow 199 \rightarrow 90 \rightarrow 58 \rightarrow 55 \rightarrow 39 \rightarrow 38 \rightarrow 18$

Total head movement:

• $100 \rightarrow 199 = 99$

• $199 \rightarrow 18 = 181$

Total = $99 + 181 = 280$ tracks

b) C-SCAN (Circular SCAN):

- Head moves in one direction (towards 199), services requests.
- Jumps to beginning @ and continues servicing.

Movement: $100 \rightarrow 150 \rightarrow 160 \rightarrow 184 \rightarrow 199 \rightarrow \text{jump to } 0 \rightarrow 18 \rightarrow 38 \rightarrow 39 \rightarrow 55 \rightarrow 58 \rightarrow 90$

Total head movement:

• $100 \rightarrow 199 = 99$

• Jump: $199 \rightarrow 0 = 199$

• $0 \rightarrow 90 = 90$

Total = $99 + 199 + 90 = 380$ tracks.

6) Consider a disk queue with requests for I/O to blocks on cylinders 98, 183, 41, 122, 14, 124, 65, 67. The FCFS scheduling algorithm is used. The head is initially at cylinder number 53. The cylinders are numbered from 0 to 199. Calculate the total head movement with respect to FCFS Algorithm, SSTF Algorithm.

A) Queue: 98, 183, 41, 122, 14, 124, 65, 67

Initial head: 53

a) FCFS (First-come, First-Served)

Movement: $53 \rightarrow 98 \rightarrow 183 \rightarrow 41 \rightarrow 122 \rightarrow 14 \rightarrow 124 \rightarrow 65 \rightarrow 67$

Total movement:

• $53 - 98 = 45$

• $98 - 183 = 85$

• $183 - 41 = 142$

• $41 - 122 = 81$

• $122 - 14 = 108$

• $14 - 124 = 110$

• $124 - 65 = 59$

• $65 - 67 = 2$ Total = 632 tracks

b) SSTF (Shortest Seek Time First)

Steps:

• $53 \rightarrow 65 (12)$

• $65 \rightarrow 67 (2)$

• $67 \rightarrow 41 (26)$

• $41 \rightarrow 14 (27)$

• $14 \rightarrow 98 (84)$

• $98 \rightarrow 122 (24)$

• $122 \rightarrow 124 (2)$

• $124 \rightarrow 183 (59)$

Total = $12 + 2 + 26 + 27 + 84 + 24 + 2 + 59 = 236$ tracks

7) Suppose a disk contains 200 tracks (0-199) and the request queue contains track no: 93, 176, 42, 148, 27, 14, 180. The current position of the read/write head is 55. Calculate the total number of track movements of read/write head Look scheduling Algorithm, C-Look scheduling Algorithm.

A) Queue: 93, 176, 42, 148, 27, 14, 180

Initial head: 55

Tracks: 0-199

a) Look:

- Head moves in one direction (up), services requests.
- Then reverses direction.

Sorted Queue: 14, 27, 42, 93, 148, 176, 180

Movement: $55 \rightarrow 93 \rightarrow 148 \rightarrow 176 \rightarrow 180 \rightarrow \text{Reverse} \rightarrow 42 \rightarrow 27 \rightarrow 14$

Total movement:

• $55 \rightarrow 180 = 125$

• $180 \rightarrow 14 = 166$

Total = $125 + 166 = 291$ tracks

b) C-Look:

Head moves in one direction (up), services all

- Jumps to lowest and continues.

Movement: $55 \rightarrow 93 \rightarrow 148 \rightarrow 176 \rightarrow 180 \rightarrow \text{jump to } 14 \rightarrow 27 \rightarrow 42$

Total movement:

• $55 \rightarrow 180 = 125$

• Jump: $180 \rightarrow 14 = 166$

• $14 \rightarrow 42 = 28$

Total = $125 + 166 + 28 = 319$ tracks

8) Summarize how RAID techniques contribute to data protection and fault tolerance, and identify common characteristics of standard RAID levels (fault tolerance, capacity overhead, performance trade-offs) while explaining why RAID 0 differs.

A) RAID (Redundant Array of Independent Disks) improves:

- Fault tolerance: Survives disk failures using redundancy.
- Performance: Parallel access across disks.
- Data protection: Uses parity (or) mirroring to recover lost data.

→ Common RAID Characteristics:

RAID level	Fault Tolerance	Capacity overhead	Performance
RAID 0	x None	No overhead	High
RAID 1	Yes	50% overhead	Good read, slow write
RAID 5	Yes	1 disk overhead	Balanced
RAID 6	Yes	2 disk overhead	Slower writes
RAID 10	Yes (mirror+stripe)	50% overhead	High performance

⇒ RAID 0 is different why:

- No redundancy: If one disk fails, all data is lost.
- Only performance-focused: Uses Striping for speed.
- Not suitable for critical data.

9) Demonstrate the trade-offs b/w RAID 0+1 and RAID 10 in failure scenarios, and evaluate which configuration provides greater resilience under different disk failure patterns.

A) RAID 0+1 (Mirror of stripes)

- First: Stripe data across disks.
- Then: Mirror the striped set.

Failure scenarios:

- If one disk in a stripe fails → entire stripe is lost.
- If one full stripe fails → mirror can recover.

→ RAID 10 (stripe of mirrors)

- First: Mirror disks.
- Then: Stripe across mirrored pairs.

Failure Scenario:

- Can survive one disk failure per mirrored pair.
- more flexible: multiple failures tolerated if not in same mirror.

Feature	RAID 0+1	RAID 10
Disk failure Tolerance	Lower	Higher
Recovery flexibility	Limited	Flexible
performance	fast	fast

10) Evaluate the read-modify-write cycle in RAID 4 when handling a random write to logical block 50 in a 4-disk array (3 data + 1 parity). Show which blocks are read and written.

A) Setup: 4-disk array $\rightarrow$ 3 data + 1 parity
write to: logical block 50.

Steps in Read-Modify-Write:

- 1) Identify target data block $\rightarrow$ e.g., block 50 on Disk 2.
- 2) Read old data from block 50
- 3) Read old parity from parity disk
- 4) Calculate new parity: $\text{new_parity} = \text{old_parity} \oplus \text{old_data} \oplus \text{new_data}$
- 5) Write new data to block 50.
- 6) Write new parity to parity disk.

Blocks Accessed:

Read:

- old data block (Disk 2, block 50)
- old parity block (parity disk)

Write:

- New data block (Disk 2, block 50)
- New parity block (parity disk)

11) Apply the Concept of inodes in a file system to explain how files and directories are managed and describe the role of file permissions in UNIX/LINUX Systems.

- A) Inodes store metadata: file size, owner, permissions, time stamps, and pointers to data blocks.
 - Directories map filenames to inode numbers.
 - Permissions (rwx for user/group/others) control access.
 - r = read, w = write, x = execute
 - Ex: `chmod 755 file` → owner: all access, others: read & execute
-

12) Construct a scenario where misconfigured permission cause unintended data modification (e.g., a shared log file overwritten by a low-privilege process) and propose OS-level remedies.

- A) Scenario: `/var/log/shared.log` is writable by all (`chmod 666`)
- A low-privilege script overwrites logs accidentally.

• Remedies:

- Use `chmod 644` → only owner can write
 - Set group ownership and use ACLs
 - Use `chattr +a` to allow only appending.
-

13) Apply your understanding of file system metadata to analyze how corruption in components such as inodes, directory entries, (or) allocation tables (e.g., FAT) can lead to data loss. Illustrate your answer with practical scenarios that show the role of metadata in maintaining file integrity.

A) FAT & Consistency Checking:

- FAT maps each file to a linked list of sectors.
 - fsck/chkdsk scan FAT and directory entries;
 - Detect orphaned blocks
 - Fix broken chains
 - Mark bad sectors
-

14) ~~Construct~~ Explain how a File Allocation Table (FAT) manages file storage, and describe the process of consistency checking in file systems with fsck or chkdsk for maintaining data integrity.

A) ~~Meta~~

- Inode corruption → file inaccessible.
 - Directory entry corruption → filename lost
 - FAT corruption → file block chain broken
 - Ex: Power failure corrupts FAT → file points to wrong blocks, data unreadable.
-

15) Demonstrate the difference between soft updates and journaling by applying both approaches to a directory update operation.

A) soft updates: Carefully order metadata writes to avoid inconsistency.

- Journaling: Log changes before applying them.
- Ex: Rename directory,
 - soft update → update inode, then directory entry
 - Journaling → write to journal, then commit

16) Demonstrate how the Network File System (NFS) facilitates remote file sharing across a network by applying its client-server architecture. Illustrate how file, access, chaining, and consistency are managed during distributed file operations.

A) Network File System - Remote File Sharing.

- Client-server model: Server exports directories, clients mount them.
- Access: client uses RPC to read/write.
- Caching: client caches reads; uses timestamp checks for consistency.
- Consistency: NFSv4 improves locking and stateful sessions.

17) Apply the concept of ECC (Error-Correcting Code) in a real-time scenario where a memory module detects and corrects a single-bit error.

- A) • Scenario: RAM detects bit flip in address 0x1A2B
- ECC identifies and corrects single-bit error using parity bits.
 - System continues without crash (or) data loss.

18) Apply the concept of digital signatures and checksums in a scenario where two systems exchange files securely over a network.

⇒ Demonstrate how tampered files can be detected using hashes.

A) ⇒ Sender: Hashes file → Signs hash with private key.

⇒ Receiver: Verifies hash with public key.

⇒ Tamper detection: If file is changed, hash won't match → Integrity failure.

19) Compare different file allocation strategies such as Sequential, Contiguous, Linked, and Indexed allocation. Illustrate each with examples and evaluate their impact on file access efficiency and fragmentation.

A)

Strategy	Example	Efficiency	Fragmentation
Sequential	Tape backup	High	None
Contiguous	Video file	Fast	High
Linked	Logs	Slow	Low
Indexed	OS files	Balanced	Moderate

20) Analyze a disk system with the following that has 32 surfaces, 64 tracks per surface, and 512 sectors per track, each sector holding 256 bytes.

Determine:

The total storage capacity is bytes

The number of bits required to uniquely address a sector.

A) • Surfaces: 32

• Tracks/surface: 64

• Sectors/track: 512

• Bytes/sector: 256

• Total capacity: $32 \cdot 64 \cdot 512 \cdot 256 = 268,435,456$

bytes = 256 MB

• Addressable sectors:

$32 \cdot 64 \cdot 512 = 1,048,576$

⇒ Bits to address: $\log_2(1,048,576) \approx 20 \text{ bits}$

- 2) Analyze a disk system with the following specifications:
- 16 platters (i.e., 32 recording surfaces)
 - 80 tracks per surface
 - 1024 sectors per track
 - Each sector stores 512 bytes.

Determine:

- The total storage capacity in gigabytes (GB).
- The number of bits required to uniquely address each sector on the disk.

A) Surfaces: 32

Tracks / Surface: 80

Bytes / Sector: 512

Total capacity:

$$32 \cdot 80 \cdot 1024 \cdot 512 = 1,342,177,280 \text{ bytes} = 1.25 \text{ GB}$$

Addressable sectors:

$$32 \cdot 80 \cdot 1024 = 2,621,440$$

$$\Rightarrow \text{Bits to address: } \log_2(2,621,440) \approx 22 \text{ bits}$$